OSI Model and TCP/IP Model

The **OSI model** splits network communication into 7 layers, each giving services to the layer above.

Layer	Job	Examples
7 Application	Services to the user	HTTP, FTP, SMTP, DNS
6 Presentation	Format, encryption, compression	TLS, JPEG
5 Session	Start, manage and end sessions	RPC
4 Transport	End-to-end delivery, ports	TCP, UDP
3 Network	Logical addressing and routing	IP, ICMP, routers
2 Data Link	Framing, MAC addresses, error detection	Ethernet, switches
1 Physical	Bits over the medium	Cables, hubs

The **TCP/IP model** used on the Internet has 4 layers: **Application** (OSI 5–7), **Transport**, **Internet** (Network) and **Network Access** (OSI 1–2).

Encapsulation: each layer adds its header as data goes down: data → segment (transport) → packet (network) → frame (data link) → bits (physical). The receiver removes the headers in reverse order.

Exam tip: Switches work at layer 2 using MAC addresses; routers work at layer 3 using IP addresses.

TCP and UDP

Point	TCP	UDP
Connection	Connection-oriented (handshake)	Connectionless
Reliability	Reliable: acknowledgements and retransmission	No guarantee of delivery
Order	Delivers bytes in order	No ordering
Speed	Slower, more overhead	Faster, less overhead
Header size	20–60 bytes	8 bytes
Control	Flow and congestion control	None
Used by	HTTP, HTTPS, FTP, SMTP, SSH	DNS, video calls, online games, DHCP

TCP three-way handshake

- Client → Server: **SYN** (seq = x)
- Server → Client: **SYN-ACK** (seq = y, ack = x + 1)
- Client → Server: **ACK** (ack = y + 1). The connection is established.

Connection close uses a **four-way handshake**: FIN, ACK, FIN, ACK. The side that closes first waits in the **TIME_WAIT** state so that delayed packets do not affect a new connection.

Ports

A port number identifies the application on a host. Well-known ports: HTTP 80, HTTPS 443, FTP 21, SSH 22, SMTP 25, DNS 53. A **socket** = IP address + port.

Exam tip: Choose UDP when speed matters more than perfect delivery, like live video, where a late packet is useless anyway.

Flow Control and Congestion Control

Flow control stops a fast sender from overwhelming a slow **receiver**. **Congestion control** stops senders from overwhelming the **network**.

Sliding window

The receiver advertises a **receive window (rwnd)**: how many bytes it can still accept. The sender can have at most that many unacknowledged bytes in flight. Error recovery uses **Go-Back-N** (resend everything from the lost frame) or **Selective Repeat** (resend only the lost frame).

TCP congestion control

- **Slow start**: the congestion window (cwnd) starts at 1 segment and doubles every round-trip time until it reaches the threshold (ssthresh).
- **Congestion avoidance**: after the threshold, cwnd grows by 1 segment per round-trip time (additive increase).
- **On timeout**: ssthresh = cwnd / 2 and cwnd goes back to 1.
- **On 3 duplicate ACKs** (fast retransmit / fast recovery): ssthresh = cwnd / 2 and cwnd = ssthresh (multiplicative decrease).

The amount the sender can send = **min(cwnd, rwnd)**.

Example: with ssthresh = 16, cwnd grows 1, 2, 4, 8, 16 in slow start, then 17, 18, 19 in congestion avoidance. If a timeout happens at 20, ssthresh becomes 10 and cwnd restarts at 1.

> **Exam tip:** AIMD (additive increase, multiplicative decrease) is what makes TCP fair between users.

IP Addressing and Subnetting

An **IPv4 address** is 32 bits, written as four decimal numbers (for example 192.168.10.77). It has a **network part** and a **host part**; the **subnet mask** or **CIDR prefix** (/24) tells where the split is. IPv6 addresses are 128 bits.

Private ranges: 10.0.0.0/8, 172.16.0.0/12 and 192.168.0.0/16. They are used inside organisations and reach the Internet through **NAT**.

Subnetting formulas

- Number of addresses in a subnet = $2^{(32 - \text{prefix})}$
- Usable hosts = $2^{(32 - \text{prefix})} - 2$ (network and broadcast addresses are reserved)

Worked example: 192.168.10.0/26

- Mask = 255.255.255.**192** (26 ones).
- Addresses per subnet = 2^6 = **64**; usable hosts = **62**.
- Subnets: 192.168.10.0, .64, .128 and .192 (4 subnets).
- For host 192.168.10.**77**: network address = **192.168.10.64**, broadcast = **192.168.10.127**, usable range = .65 to .126.

Example: an office needs subnets of at least 50 hosts. $2^6 - 2 = 62 \geq 50$, so a /26 prefix fits.

> **Exam tip:** To find the network address, AND the IP address with the subnet mask.

Routing Algorithms

Routing finds the path a packet takes from source to destination. Routers build **routing tables** and forward each packet to the next hop.

Point	Distance Vector	Link State
Knowledge	Only neighbours' distance tables	Full map of the network
Algorithm	Bellman–Ford	Dijkstra's shortest path
Updates	Periodic, whole table to neighbours	Only on changes, flooded to all
Convergence	Slow; count-to-infinity problem	Fast
Protocol	RIP (max 15 hops)	OSPF

Count-to-infinity

In distance vector routing, when a link fails, routers can keep increasing their distance through each other. Fixes: **split horizon** (don't advertise a route back to the neighbour you learned it from), **poison reverse** and a maximum hop count.

Dijkstra example

Links: A–B = 4, A–C = 1, C–B = 2, B–D = 1, C–D = 5. Shortest paths from A: C = 1, B = 3 (A–C–B), D = 4 (A–C–B–D).

Interior protocols (RIP, OSPF) work inside one organisation. **BGP** is the exterior protocol that connects organisations (autonomous systems) across the Internet.

> **Exam tip:** Link state converges faster but uses more memory and CPU, because every router stores the whole map.

DNS and How a Web Page Loads

The **Domain Name System** translates names like www.example.com into IP addresses. It is a distributed, hierarchical database: **root servers**, **top-level domain (TLD) servers** (.com, .in) and **authoritative servers** for each domain. DNS mainly uses UDP port 53.

Resolution steps

- The browser checks its cache, then the operating system cache.
- The query goes to the **local (recursive) resolver**, usually run by the ISP.
- The resolver asks a root server, which points to the .com TLD server.
- The TLD server points to the authoritative server for example.com.
- The authoritative server returns the IP address, which is cached for its **TTL**.

Record types: **A** (IPv4 address), **AAAA** (IPv6), **CNAME** (alias), **MX** (mail server), **NS** (name server).

Loading a web page

DNS lookup → TCP three-way handshake with the server → TLS handshake for HTTPS (agree on keys, verify the certificate) → HTTP GET request → the server responds with HTML → the browser fetches CSS, JavaScript and images, and renders the page.

Exam tip: Recursive: the resolver does all the work for the client. Iterative: each server only points to the next server to ask.

HTTP, HTTPS and Network Delays

HTTP is a stateless request–response protocol. Methods: **GET** (read), **POST** (create), **PUT** (replace), **PATCH** (partly update), **DELETE**. Status codes: 2xx success (200 OK, 201 Created), 3xx redirect (301), 4xx client error (400, 401, 403, 404), 5xx server error (500, 503). State is kept with **cookies** or tokens.

HTTPS = HTTP over **TLS**. It gives encryption, integrity and server authentication using certificates. The TLS handshake uses asymmetric (public-key) cryptography to agree on a symmetric session key, which then encrypts the data.

Delays in a network

- **Transmission delay** = packet size / bandwidth
- **Propagation delay** = distance / signal speed
- **Queuing delay**: time waiting in router buffers
- **Processing delay**: time to check headers and choose the next hop

Example: a 1000-byte packet on a 1 Mbps link over 2000 km (signal speed 2×10^8 m/s). Transmission = 8000 bits / 10^6 bps = **8 ms**. Propagation = 2×10^6 m / 2×10^8 m/s = **10 ms**. Total (ignoring queuing and processing) = **18 ms**.

Bandwidth–delay product = bandwidth × round-trip time: the number of bits that fit 'in the pipe'. It decides the window size needed to use a link fully.

Exam tip: Bigger bandwidth reduces transmission delay but not propagation delay; only a shorter distance reduces that.